

Tugas Besar 3

IF2211 Strategi Algoritma

Implementasi Pattern Matching pada Chromium Browser Extension
untuk Deteksi Judi Online

Semester 2 Tahun 2025/2026



Disusun oleh:

SL0T999

Vincent Rionarlie	(13524031)
Jason Edward Salim	(13524034)
Muhammad Aufar Rizqi Kusuma	(13524061)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung



Daftar Isi

1	Pendahuluan	2
1.1	Latar Belakang	2
1.2	Spesifikasi Wajib	3
1.3	Persyaratan dan Larangan	5
1.4	Spesifikasi Bonus	6
2	Landasan Teori	7
3	Analisis Pemecahan Masalah	8
4	Implementasi dan Pengujian	9
5	Penutup	10
5.1	Kesimpulan	10
5.2	Saran	10
6	Lampiran	11
6.1	Pernyataan Tidak Melakukan Kecurangan	11
6.2	Tautan GitHub	11
6.3	Tabel Checklist Spesifikasi	12
6.4	Tabel Kontribusi	13

Bab 1

Pendahuluan

1.1 Latar Belakang

Luffy adalah seorang mahasiswa Teknik Informatika yang sedang menjalani semester akhirnya bersama timnya. Setelah mempelajari berbagai materi selama masa perkuliahan, mereka mendapatkan tugas besar terakhir sebagai bentuk penerapan ilmu yang telah dipelajari. Luffy ingin membuat sebuah proyek yang tidak hanya memenuhi kebutuhan akademik, tetapi juga memiliki hubungan dengan permasalahan yang sering ditemui di kehidupan sehari-hari.



Gambar 1.1: Luffy mengerjakan tugas (Sumber: AI Generated Image)

Luffy adalah seorang mahasiswa Teknik Informatika yang sedang menjalani semester akhirnya bersama timnya. Setelah mempelajari berbagai materi selama masa perkuliahan, mereka mendapatkan tugas besar terakhir sebagai bentuk penerapan ilmu yang telah dipelajari. Luffy ingin membuat sebuah proyek yang tidak hanya memenuhi kebutuhan akademik, tetapi juga memiliki hubungan dengan permasalahan yang sering ditemui di kehidupan sehari-hari.

Saat menggunakan internet untuk belajar, mencari referensi, maupun membuka berbagai website, Luffy sering menemukan kemunculan iklan dan konten perjudian online yang tersebar di berbagai halaman web. Tidak jarang, konten tersebut

menggunakan variasi penulisan yang unik dan tersamarkan agar sulit dikenali secara langsung. Dari situ, Luffy mulai tertarik untuk mempelajari bagaimana sebuah sistem dapat mengenali pola-pola tertentu pada teks secara otomatis dan efisien.

Berangkat dari ide tersebut, Luffy dan timnya memutuskan untuk mengembangkan sebuah browser extension bernama Judol Detector sebagai proyek tugas besar mereka. Melalui proyek ini, mereka berharap dapat menggabungkan konsep algoritma, pemrosesan string, dan pengembangan perangkat lunak ke dalam sebuah aplikasi yang interaktif dan aplikatif.



Gambar 1.2: Visualisasi Browser Extension (Sumber: Asisten IRK)

1.2 Spesifikasi Wajib

Pada tugas besar ini, buatlah sebuah chromium browser extension yang dapat mendeteksi informasi yang mengandung unsur judol. Sebagai catatan, seperti yang biasa kita temukan dalam website sekarang dengan format `<kata><angka>`. Contoh: GACOR99, MAXWIN88, MADU308, dan sebagainya. Sistem harus memiliki minimal fitur dengan detail seperti berikut.

1. Extension yang dibuat pada tugas besar ini dikembangkan dengan bahasa Typescript. Dalam proses pencocokan kata kunci, sistem wajib mengimplementasikan algoritma Knuth-Morris-Pratt dan Boyer Moore serta RegEx. Untuk algoritma selain RegEx, pencarian dilakukan menggunakan file `keyword.txt` (file ini boleh dimodifikasi sesuai kebutuhan) yang dipisahkan dengan newline untuk melakukan string matching secara iterative, sehingga pengguna tidak memasukkan input, tapi sistem yang membaca file tersebut secara langsung.

Untuk pencarian dengan regex, format pencarian seminimal mungkin dapat handle 2 atau 3 angka di belakang kata tanpa spasi. Contoh: MAXWIN234, SLOT99. Test case yang diberikan nanti akan sangat bervariasi, sehingga pastikan pencarian RegEx diimplementasikan dengan detail dan dapat handle banyak edge case. Adapun juga algoritma Aho-Corasick dan Rabin Karp apabila ingin mengerjakan bonus, sesuai tertera di Spesifikasi Bonus dibawah.

2. Perlu diketahui bahwa, Developer Judol pun juga membuat penamaan dari websitenya, seperti pada teknik Phishing, jadi membuat nama domain yang serupa dengan pada umumnya, namun nyatanya berbeda. Ini terjadi oleh

karena adanya perbedaan kode komputer (unicode), dan komputer mengiranya adalah karakter yang berbeda.

Maka dari itu, sistem harus bisa membaca kecocokan secara exact matching dan juga fuzzy matching (apabila tidak ditemukan kecocokan yang persis). Untuk setiap kata kunci yang tidak memiliki kemunculan sama sekali pada saat exact matching, lakukan pencarian kembali dengan perhitungan tingkat kemiripan menggunakan algoritma Weighted Levenshtein Distance. Penyesuaian bobot (weight) ini krusial karena pelaku sering kali menyiasati strategi dengan menggunakan huruf pengganti yang mirip secara visual (misalnya mengubah huruf 'O' menjadi angka '0', 'A' menjadi '4', atau 'I' menjadi '1'). Implementasi harus memperhatikan hal-hal berikut:

- (a) Weighted Cost: Berikan bobot penalti yang lebih kecil untuk substitusi karakter yang mirip secara visual dibandingkan dengan karakter yang benar-benar berbeda.
- (b) Thresholding: Tentukan nilai ambang batas (threshold) kemiripan yang tepat untuk memastikan sistem tetap memberikan hasil yang relevan namun tetap menghindari munculnya false positive.

Algoritma ini memungkinkan sistem untuk tetap menampilkan hasil pencarian yang relevan meskipun terdapat perbedaan minor, kesalahan ketik, atau upaya manipulasi teks oleh pengguna. Perlu diperhatikan bahwa secara konsep, exact matching juga merupakan bagian dari fuzzy matching.

$$\text{lev}(a, b) = \begin{cases} |a| & \text{if } |b| = 0, \\ |b| & \text{if } |a| = 0, \\ \text{lev}(\text{tail}(a), \text{tail}(b)) & \text{if } \text{head}(a) = \text{head}(b), \\ 1 + \min\{ \text{lev}(\text{tail}(a), b), \\ \quad \text{lev}(a, \text{tail}(b)), \\ \quad \text{lev}(\text{tail}(a), \text{tail}(b)) \} & \text{otherwise.} \end{cases}$$

3. Jadi sistem menjalankan pengecekan dengan hirarki sebagai berikut:

- Exact Matching: Algoritma pattern matching untuk mencari kata kunci dasar dari **keyword.txt**.
 - RegEx Matching: Secara paralel atau sekuensial, jalankan Regex untuk menangkap pola <kata><angka>.
 - Fuzzy Matching: Kalau string dalam DOM tidak lolos namun memiliki kemiripan, jalankan dengan Weighted Levenshtein Distance dalam upaya manipulasi karakter
4. Apabila ditemukan result keyword, maka elemen DOM terkait harus ditandai, dengan warna bebas. Namun highlight tidak boleh merusak layout, tepat sasaran dan dapat terhapus saat rescanning.

5. Saat user hover pada elemen yang terdeteksi, maka tampilkan tooltip dengan informasi berikut:

- Keyword yang terdeteksi
- Algoritma yang digunakan
- Jumlah kemunculan
- Waktu eksekusi

Tooltip dibuat dengan custom DOM element.

6. Terdapat juga statistik secara realtime pada popup extension yang menampilkan:

- Total keyword yang ditemukan
- Perbandingan jumlah keyword yang satu dengan lainnya, menggunakan visualisasi
- Waktu eksekusi tiap algoritma
- Jumlah match tiap algoritma

7. Setiap kelompok harap mengisi nama kelompok dan anggotanya pada link berikut, paling lambat Minggu, 17 Mei 2026 23.59. Nama kelompok dilarang mengandung unsur SARA, penghinaan, provokasi, atau hal-hal lain yang bertentangan dengan norma hukum.

Perlu diperhatikan bahwa kelompok tubes untuk mata kuliah ini tidak boleh sama. Jadi jika berkelompok dengan X pada tubes ini, maka pada tubes selanjutnya anda tidak diperkenankan berkelompok dengan X.

1.3 Persyaratan dan Larangan

- Seluruh algoritma string matching wajib dibuat from scratch
- Dilarang keras menggunakan
 - `string.includes()`
 - `string.indexOf()`
 - Built-in search function
 - External string matching library
- Implementasi manual wajib mencakup:
 - Border function
 - Failure function
 - Last occurrence table
 - Shifting process
 - Comparison Counting
- Regex diperbolehkan menggunakan regex engine dari JavaScript.

1.4 Spesifikasi Bonus

1. (6 poin) **[Video]** Membuat video tentang browser extension Judol Detector yang dikembangkan, kemudian mengunggahnya di Youtube. Video dibuat harus memiliki audio dan menampilkan wajah dari setiap anggota kelompok. Untuk contoh video tubes stima tahun-tahun sebelumnya dapat dilihat di Youtube dengan kata kunci “Tubes Stima”, “strategi algoritma”, “Tugas besar stima”, dll. Semakin menarik dan informatif video, maka semakin banyak poin yang diberikan.
2. (4 poin) **[Algoritma Bonus]** Mengimplementasikan Algoritma Aho-Corasick dan Rabin Karp sebagai algoritma alternatif.
3. (5 poin) **[Censorship / Blur Teks]** Mengimplementasikan fitur blur visual secara otomatis pada elemen halaman web yang terdeteksi memuat konten judi online. Tidak hanya sekadar menghighlight teks, ekstensi langsung melakukan aksi blur pada elemen yang mengandung kata kunci judol, sehingga konten tersebut tidak terlihat oleh pengguna. Perlu diperhatikan, terdapat toggle on/off untuk ini.
4. (5 poin) **[Optical Character Recognition Pada Gambar]** Mengimplementasikan fitur OCR untuk mendeteksi kata kunci judi online yang sengaja disembunyikan dalam bentuk gambar. Ekstensi akan mengekstrak teks dari elemen gambar yang ditemukan di halaman, kemudian mencocokkannya dengan daftar kata kunci judol, lalu melakukan blur atau penggantian pada gambar yang terdeteksi mengandung konten judol. Fitur ini dapat diimplementasikan dengan Tesseract.js



Bab 2

Landasan Teori



Bab 3

Analisis Pemecahan Masalah



Bab 4

Implementasi dan Pengujian



Bab 5

Penutup

5.1 Kesimpulan

5.2 Saran

Bab 6

Lampiran

6.1 Pernyataan Tidak Melakukan Kecurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Vincent Rionarlie
13524031

Jason Edward Salim
13524034

Muhammad Aufar
Rizqi Kusuam
13524061

6.2 Tautan GitHub

Link GitHub

6.3 Tabel Checklist Spesifikasi

No	Poin	Ya	Tidak
1	Extension berhasil di-build dan di-load tanpa kesalahan pada Chromium browser dan dikembangkan dengan TypeScript		
2	KMP dan Boyer-Moore diimplementasikan from scratch		
3	Regex meng-handle format <kata><angka>dan berbagai edge case		
4	Pencarian KMP & BM membaca 'keywords.txt' secara iteratif dan tidak menggunakan built-in search function atau library eksternal		
5	Exact matching dan fuzzy matching berjalan benar		
6	Elemen DOM terdeteksi diberi highlight dan terhapus saat rescanning		
7	Tooltip muncul saat hover dengan informasi keyword, algoritma, kemunculan, dan waktu eksekusi		
8	Popup menampilkan statistik realtime (total keyword, perbandingan, waktu eksekusi, jumlah match)		
9	[Bonus] Membuat Video YouTube (jika dikerjakan)		
10	[Bonus] Implementasi Algoritma Aho-Corasick dan Rabin-Karp		
11	[Bonus] Implementasi Censorship / Blur Teks dengan toggle on/off		
12	[Bonus] Implementasi Optical Character Recognition pada Gambar		

6.4 Tabel Kontribusi

No	Nama Lengkap	NIM	Deskripsi Pekerjaan	Persentase Kontribusi (%)
1	Vincent Rionarlie	13524031	Pembuatan algoritma pemrosesan string, bonus algoritma Aho-Corasick dan Rabin-Karp, laporan akhir	34
2	Jason Edward Salim	13524034	Pembuatan FE beserta dengan build-up nya, bonus censoring, laporan akhir	33
3	Muhammad Aufar Rizqi Kusuam	13524061	Pembuatan Controller program (Scanner, highlighter, DOM Integration, dll), laporan akhir	33



Bibliografi